Document Number: P3841R0
Date: 2025-09-12
Audience: Library Evolution Working Group Incubator
Reply-To: Avi Kivity <avi@scylladb.com>

Proposal for `std::constructor` Function Object

I. Motivation

C++ users can convert member functions to funtion objects with `std::mem_fn()`, partially bind arguments to functions or function objects with `std::bind()`, `std::bind_front()`, and `std::bind_back()`, type-erase them with `std::function<>`, and use them to transform ranges with `std::views::transform()`. But none of that can be done directly to class constructors; a helper function must be used to "downgrade" the constructor into a function.

The proposed `std::constructor<T>()` is a utility function object that provides a convenient, generic mechanism to convert a constructor overload set into a function object, thereby allowing all the existing tooling for function objects to be brought to bear on it.

II. Example Problem

Imagine you have a range of `size_t` and you wish to return a vector of vectors, with the sizes given from the given range. Naive code can look like:

```
    std::vector<std::vector<int>> result;
    result.reserve(std::distance(input));
    for (auto sz : input) {
         result.emplace_back(sz);
    }
```

However, this is unsatisfying. The input range may be an input_range, which does not afford two passes (one for std::distance, one for the for loop). The emplace_back loop is less efficient than constructing the vector from a range.

A modern range-based solution would look like

```
   auto result = input
       | std::views::transform([] (size_t sz) {
           return std::vector<int>(sz);
       }
       | std::ranges::to<std::vector>();
```

This is still unsatisfying, as the lambda is not concise.

We propose `std::constructor<T>()`, similar to `std::mem_fn()` but instead of converting a member function to a callable object, it converts a constructor overload set to a callable object. With std::constructor, the example above can be written as

```
    auto result = input
        | std::views::transform(std::constructor<std::vector<int>>())
        | std::ranges::to<std::vector>();
```

III. Proposed Solution: `std::constructor`

A. Function Signature

`std::constructor<T>()` evaluates to a function object that perfectly forwards its arguments to T's constructors.

```
namespace std {

    template <typename T>
    struct constructor {
        template <typename... Args>
```

```
        static constexpr T operator()(Args&&... args)
                noexcept(std::is_nothrow_constructible_v<T, Args...>) {
            return T(std::forward<Args>(args)...);
        };
    };

}
```

B. Semantics

- Constructs an object of type `T` using perfect forwarding
- Supports any public constructor of `T`
- Returns the constructed object by value
- Works with both trivial and complex types
- `constexpr`-compatible
- `noexcept` preserving
- if `T` is a reference type and `std::constructor<T>()` attempts to bind its return value to a temporary, the program is ill formed

IV. Example Usage

```
// Basic usage (not expected in common programs)
auto str = std::constructor<std::string>()("Hello");

// Complex type construction
struct Complex {
    int x, y;
    Complex(int a, int b) : x(a), y(b) {}
};
auto comp = std::constructor<Complex>()(10, 20);

// Composability with std::bind_front
auto make_imag = std::bind_front(std::constructor<Complex>(), 0);
auto sqrt_minus_one = make_imag(1);

// Updated example from above
auto input = std::views::iota(0, 10);
auto result = input
    | std::views::transform(std::constructor<std::vector<int>>())
    | std::ranges::to<std::vector>();

// Bind an allocator to a container constructor
auto make_vector_with_alloc = std::bind_back(std::constructor<std::vector<int>>(), std::ref(alloc));
```

V. Design Considerations

Advantages

- Type-safe
- No memory allocation overhead
- Works with any constructible type
- Composable with the rest of the functional library

Potential Concerns

- Might be seen as redundant with P3312

Naming

`std::constructor<T>()` is seen as consistent with `std::plus<T>()`.

Other alternatives:

- `std::make_obj_using_allocator()` is similar. Perhaps `std::make_obj<T>()` or `std::make_object<T>()` would work.
- `std::make_from_tuple()` is also similar. So perhaps `std::make<T>(...)`?!

VI. Implementation

Reference implementation:

```
template <typename T>
struct constructor {
    template <typename... Args>
    static constexpr T operator()(Args&&... args)
            noexcept(std::is_nothrow_constructible_v<T, Args...>) {
        return T(std::forward<Args>(args)...);
    };
};
```

VII. Wording

TBD

VIII. Complexity

- Time Complexity: O(1) construction time
- Space Complexity: No additional space overhead

IX. Proposed Standardization

Recommend inclusion in the `<functional>` header in a future C++ standard revision.

X. Acknowledgments

Thanks to Arthur O'Dwyer for correcting an ealier version on the mailing list, and to Claude with assistance on this draft.